LOS ASSIGNMENT – 01

Linux Operating System

1. What If There Were No Operating System?

   An operating system is system software that acts as an interface between users, application and hardware. Without an operating system, a computer would be very difficult to use because users have to interact directly with the hardware.

   Problems Without OS and Solutions With OS

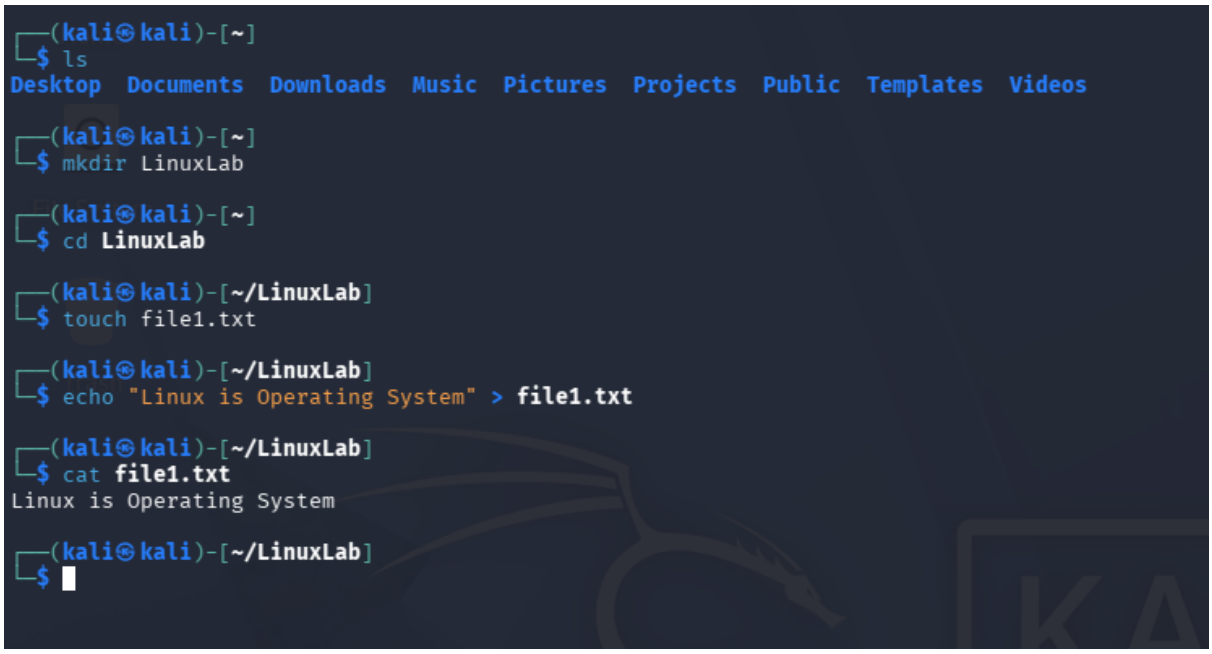
Problem Without OS	Solution Provided by OS
Hardware must be controlled manually	OS manages hardware resources
Programs cannot be easily executed	OS manages process execution
Files are difficult to create and manage	OS provides file systems and file-management tools
Devices are difficult to access	OS uses device drivers
No proper user security	OS provides users, groups and permissions

   Without OS vs With OS Comparision

Feature	Without OS	With OS
Running program	Very difficult	Easy
File management	Manual and complicated	Simple commands and file systems
Memory management	Manual	Automatically managed
Security	Very limited	Authentication and permissions
Networking	Direct hardware-level control	Built-in networking support

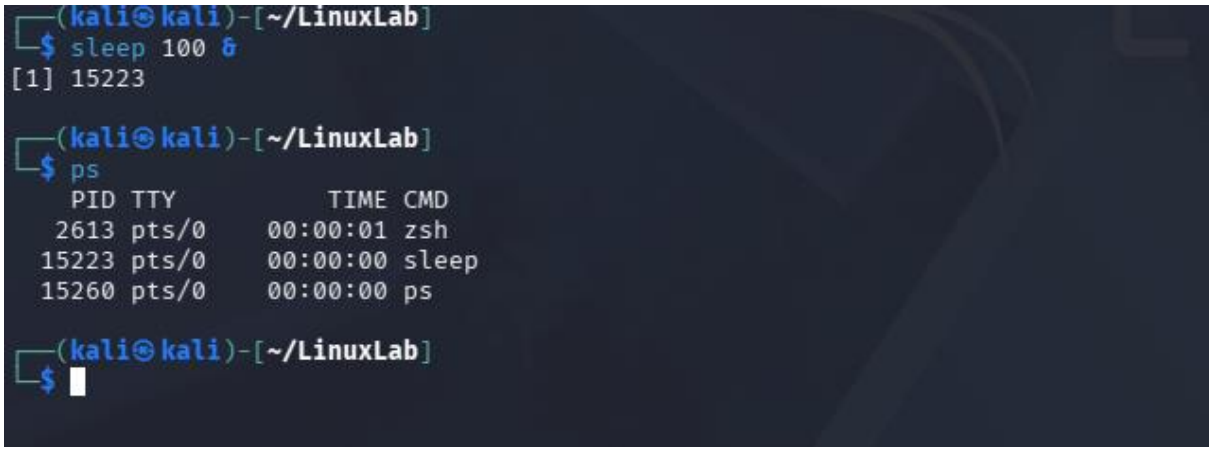
Features of Linux Command

Feature 1: File management



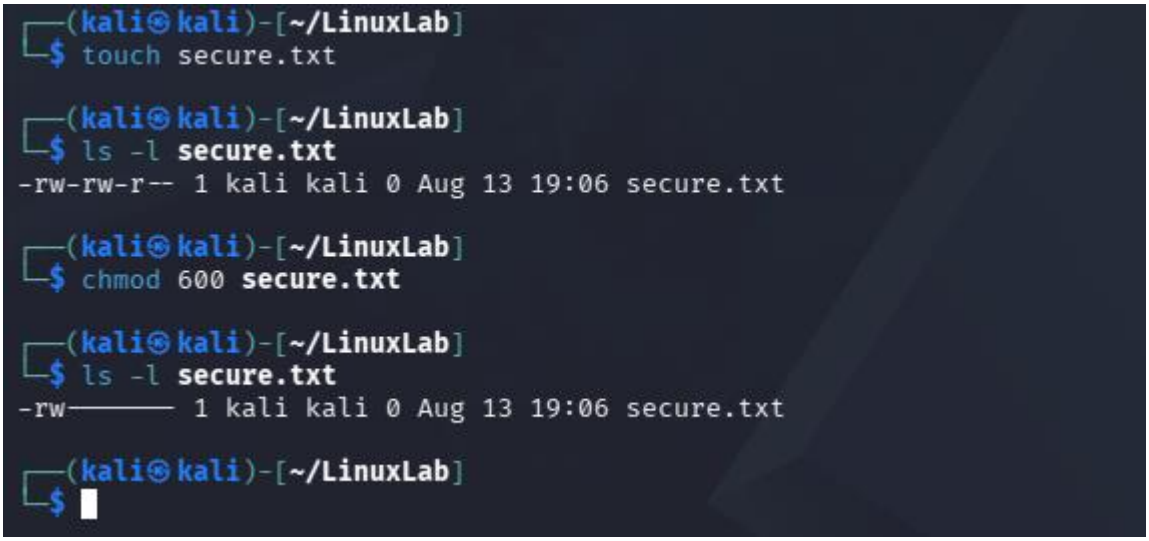
Linux provides commands for creating directories, creating files, writing data, and displaying file contents.

Feature 2: Process Management



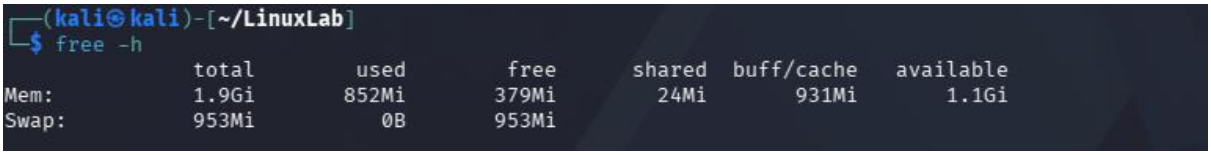
Linux creates and manages processes and allows users to view running processes.

Feature 3: File Permissions



chmod 600 gives the owner read and write permission while preventing access by other users.

Feature 4: Memory Management



This command displays information about memory usage.

2. Imagine your team is setting up a cybersecurity laboratory. Explore at least five Linux distributions relevant to cybersecurity and recommend suitable distributions for different roles such as penetration testing, digital forensics, privacy, and security monitoring. Present your findings creatively as a comparison chart, poster, infographic, or “OS selection guide.”

Linux Distribution	Main Purpose	Recommended Role
Kali Linux	Penetration testing and security assessment	Penetration Testing
Parrot Security OS	Security testing and privacy	Ethical Hacking and Security Research
Tails	Privacy and anonymity	Privacy
CAINE	Digital forensics	Digital forensics

REMnux	Malware analysis	Malware analysis
Security Onion	Network security monitoring	SOC and Security Monitoring

1. **Kali Linux**

Kali Linux is designed for penetration testing and security auditing. It includes tools such as Nmap, Metasploit, Wireshark, Burp Suite, etc.

Recommended for: Penetration Testing and ethical hacking.

2. **Parrot Security OS**

Parrot Security OS provides penetration-testing, security, privacy and development tools in a lightweight environment.

Recommended for: Ethical hacking, security research and privacy.

3. **Tails**

Tails is a privacy-focused Linux distribution that uses the Tor network and is designed to minimize traces on the computer.

Recommended for: Privacy and anonymous security research.

4. **CAINE**

CAINE is designed for computer forensics and incident-response investigations. It provides tools for evidence collection and forensic analysis.

Recommended for: Digital forensics and incident response.

5. **REMnux**

REMnux is specialized for reverse engineering and malware analysis.

Recommended for: Malware analysis and reverse engineering.

6. Security Onion

Security Onion is designed for defensive security, network monitoring, intrusion detection and threat hunting.

Recommended for: SOC operations and security monitoring.

OS Selection Guide

Cybersecurity Role	Best Distribution
Penetration Testing	**Kali Linux**
Ethical Hacking	**Kali Linux/ Parrot Security**
Privacy	**Tails**
Digital Forensics	**CAINE**
Malware Analysis	**REMnux**
Security Monitoring	**Security Onion**

3. Execute at least five Linux commands that produce both successful and unsuccessful results. Investigate their exit status using `$?`.

   Create a simple visual/table showing *Command → Result Exit Status → Meaning*.

   What can a script learn from an exit status?

   An exit status is a numerical value returned by a command after it finishes execution.

- 0 = Successful execution
- Non-zero value = Failure or another condition

The echo $? contains the exit status of the most recently executed command.

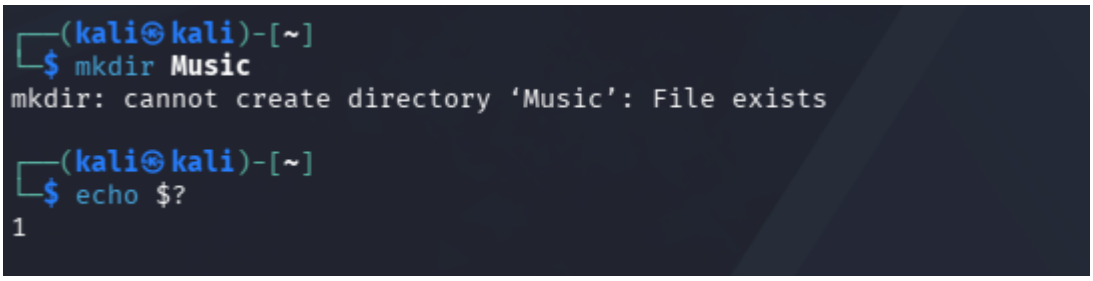
Command	Result	Exit Status	Meaning
pwd	Shows current directory	0	Successful

echo "Hello”	Prints Hello	0	Successful
mkdir exit_demo	Creates directory	0	Successful
ls /folder_that_does_not_exist	Shows an error	2	Directory does not exist
cat missingfile.txt	Shows an error	1	File cannot be read

Demonstration:

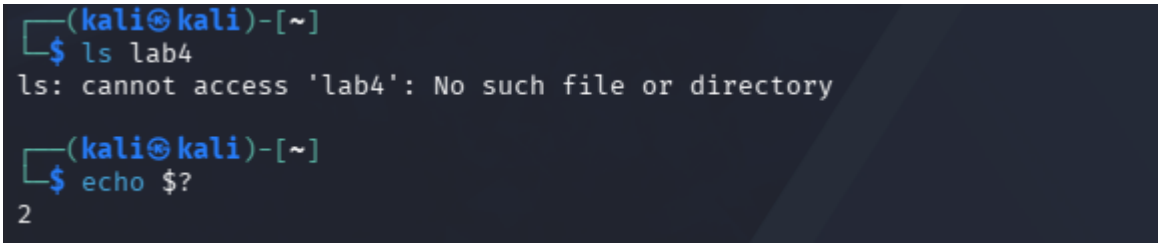
1. **Successful mkdir-**

Unsuccessful mkdir-

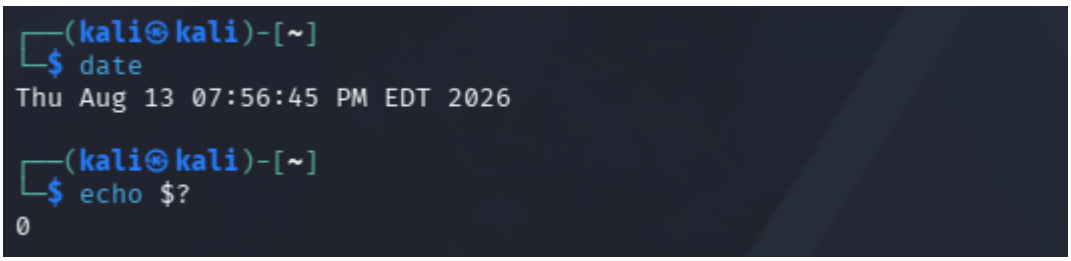


2. **Successful ls-**

Unsuccessful ls-



3. **Successful date-**



Unsuccessful date-

```
┌──(kali㉿kali)-[~]
└─$ date -d
date: option requires an argument -- 'd'
Try 'date --help' for more information.

┌──(kali㉿kali)-[~]
└─$ echo $?
1
```

4. **Successful cd-**

```
┌──(kali㉿kali)-[~]
└─$ cd aansh

┌──(kali㉿kali)-[~/aansh]
└─$ echo $?
0
```

Unsuccessful cd-

```
┌──(kali㉿kali)-[~/aansh]
└─$ cd aansh3
cd: no such file or directory: aansh3

┌──(kali㉿kali)-[~/aansh]
└─$ echo $?
1
```

5. **Successful touch-**

```
┌──(kali㉿kali)-[~/aansh]
└─$ touch file.txt

┌──(kali㉿kali)-[~/aansh]
└─$ echo $?
0
```

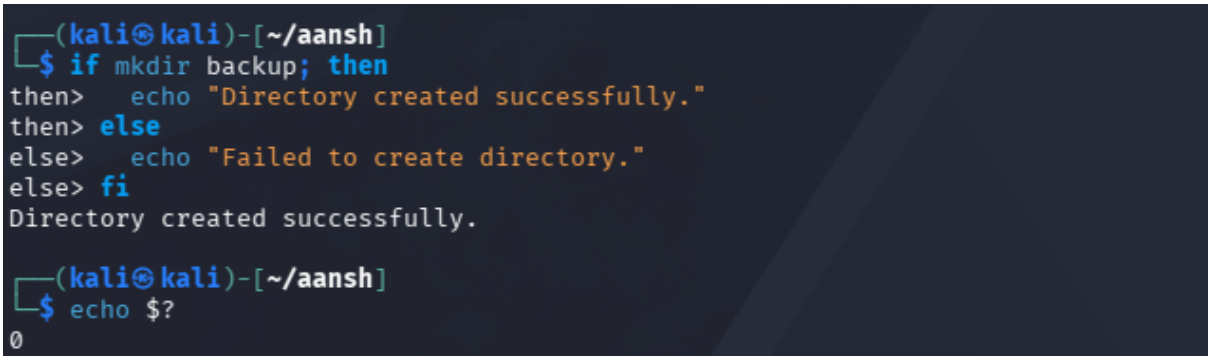
Unsuccessful touch-

```
┌──(kali㉿kali)-[~/aansh]
└─$ touch lab4/file.txt
touch: cannot touch 'lab4/file.txt': No such file or directory

┌──(kali㉿kali)-[~/aansh]
└─$ echo $?
1
```

- What Can a Script Learn From an Exit Status?

  A script can use exit status to determine whether a command succeeded or failed. It can then make decisions, handle errors, stop execution, continue execution, or display appropriate messages.



  In this, the script checks whether mkdir was successful and performs the appropriate action.